

Fase 12 — Particionamiento de Tablas en MariaDB

Materia

BDD

Continuación directa de la Fase 11. Los tres nodos activos tienen MariaDB 10.11 instalado y funcionando: `bdd-nodo01` es el maestro de replicación física, `bdd-nodo02` su esclavo (`read_only = ON`) y `bdd-nodo03` es el nodo de replicación lógica multi-maestro. Los snapshots `fase11-completa` están tomados en los tres nodos. En esta fase **toda la actividad DDL ocurre en** `bdd-nodo01` (los cambios se propagarán automáticamente a `bdd-nodo02` vía replicación física): se creará el esquema `lab_particiones` donde se demostrarán y validarán los cuatro tipos de **particionamiento nativo de MariaDB** (RANGE, LIST, HASH y KEY), que son la base conceptual y técnica directa de la fragmentación distribuida de las Fases 13 a 15.

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Distinguir entre **particionamiento nativo** (datos en un único servidor, divididos en segmentos internos) y **fragmentación distribuida** (datos en servidores físicamente separados), comprendiendo que el primero es la base conceptual del segundo.
2. Diseñar y crear tablas con los cuatro tipos de particionamiento de MariaDB: **RANGE, LIST, HASH y KEY**, incluyendo sus variantes `COLUMNS`.
3. Explicar y demostrar la **poda de particiones** (*partition pruning*): cómo el optimizador de MariaDB elimina particiones irrelevantes del plan de ejecución cuando la cláusula `WHERE` contiene un predicado sobre el atributo de particionamiento.
4. Leer e interpretar `INFORMATION_SCHEMA.PARTITIONS` para auditar la distribución de filas entre particiones y detectar desbalances.
5. Ejecutar las operaciones de **mantenimiento** de particiones: `ADD PARTITION`, `DROP PARTITION`, `REORGANIZE PARTITION`, `TRUNCATE PARTITION` y `ANALYZE PARTITION`.

6. Identificar y explicar la **incompatibilidad entre claves foráneas y particionamiento nativo** en MariaDB, y discutir cómo la fragmentación distribuida de las Fases 13–15 aborda esa limitación de forma diferente.
 7. Aplicar la restricción de **clave primaria compuesta** que impone MariaDB cuando el atributo de particionamiento no es la columna de identidad principal.
 8. Conectar el diseño de particionamiento LIST por `region` de esta fase con el plan de fragmentación horizontal del Documento de Diseño Distribuido (Fase 9), evidenciando la continuidad conceptual entre ambos enfoques.
-

B. Conceptos teóricos necesarios

1. Particionamiento nativo vs. fragmentación distribuida.

Ambas técnicas dividen los datos de una tabla en subconjuntos más pequeños, pero difieren en su ámbito físico:

Característica	Particionamiento nativo (Fase 12)	Fragmentación distribuida (Fases 13–15)
Ubicación de los datos	Un único servidor	Múltiples servidores
Transparencia	Total para el cliente	Lograda mediante el coordinador Spider
Claves foráneas	No soportadas entre tablas particionadas	Gestionadas a nivel de aplicación
Objetivo principal	Rendimiento y mantenimiento	Escalabilidad horizontal y disponibilidad
Sintaxis	<code>PARTITION BY</code> en <code>CREATE TABLE</code>	Tablas tipo Spider en el coordinador

El particionamiento nativo es el primer paso para entender la fragmentación: los mismos predicados (`region = 'norte'`, rango de fechas) que definen particiones locales en esta fase definirán fragmentos en nodos separados en las Fases 13 y 14.

2. Tipos de particionamiento en MariaDB.

- **RANGE:** divide las filas según rangos de valores de una expresión numérica o de fecha. Cada partición recibe las filas cuya expresión es menor que el límite superior (`VALUES LESS THAN`).

SQL

```
PARTITION BY RANGE (YEAR(fecha_pedido)) (  
    PARTITION p2024 VALUES LESS THAN (2025),  
    PARTITION p2025 VALUES LESS THAN (2026),  
    PARTITION p_futuro VALUES LESS THAN MAXVALUE  
)
```

- **LIST:** divide las filas según un conjunto de valores discretos y explícitos. Si un valor insertado no pertenece a ninguna lista, la operación falla con `ERROR 1526`.

SQL

```
PARTITION BY LIST COLUMNS (region) (  
    PARTITION p_norte VALUES IN ('norte'),  
    PARTITION p_sur VALUES IN ('sur')  
)
```

- **HASH:** aplica una función de módulo sobre una expresión numérica para distribuir filas uniformemente. El número de particiones se fija en la definición; no existe un predicado de negocio explícito.

SQL

```
PARTITION BY HASH (id) PARTITIONS 4
```

- **KEY:** similar a HASH pero usa el algoritmo de hashing interno de MariaDB, con soporte para columnas de cualquier tipo (incluidas cadenas de texto). Si no se especifica columna, usa la clave primaria.

SQL

```
PARTITION BY KEY (sku) PARTITIONS 4
```

3. Variantes COLUMNS.

Las variantes `RANGE COLUMNS` y `LIST COLUMNS` amplían `RANGE` y `LIST` para soportar columnas de tipo `VARCHAR`, `CHAR`, `DATE`, `DATETIME` y `ENUM`, eliminando la restricción de usar solo expresiones numéricas enteras. Son las variantes **obligatorias** para la columna `region ENUM(...)` de este laboratorio: intentar usar `LIST` estándar con un `ENUM` produce un error de sintaxis.

4. Poda de particiones (*partition pruning*).

Cuando una consulta incluye en su `WHERE` un predicado sobre el atributo de particionamiento, el optimizador identifica cuáles particiones pueden contener filas que cumplan ese predicado y descarta las demás *antes de leer ningún dato*. La poda reduce el volumen real de datos que el motor examina.

`EXPLAIN` muestra en la columna `partitions` exactamente qué particiones se consultarán. Comparar el valor de `rows` entre una consulta con y sin filtro cuantifica el beneficio de la poda:

SQL

```
-- Solo leerá p_norte (poda activa)
EXPLAIN SELECT * FROM clientes_list4 WHERE region = 'norte';
-- Leerá las cuatro particiones (sin poda)
EXPLAIN SELECT * FROM clientes_list4;
```

La poda funciona con `RANGE` y `LIST`, pero **no con `HASH` ni `KEY`**: en estos tipos el motor desconoce de antemano en qué partición cae un valor de negocio dado (solo conoce el módulo o el hash del propio atributo de particionamiento).

5. Restricción de clave primaria con particionamiento.

MariaDB exige que el atributo de particionamiento esté incluido en *cada índice único* de la tabla, incluida la clave primaria. Si la tabla tiene un `AUTO_INCREMENT` sobre una columna distinta al atributo de particionamiento, la

solución estándar es una PK compuesta:

SQL

```
PRIMARY KEY (id, region)  -- id AUTO_INCREMENT, region es la clave  
de particionamiento
```

Consecuencia importante: con una PK compuesta, el `AUTO_INCREMENT` opera de forma **por partición** en ciertas versiones del motor, lo que significa que dos filas en particiones distintas podrían recibir el mismo valor de `id`. Para el laboratorio, donde los datos se insertan con IDs explícitos, esto no representa un problema práctico, pero es esencial documentarlo para aplicaciones reales.

6. Incompatibilidad con claves foráneas.

MariaDB no permite claves foráneas (`FOREIGN KEY`) en tablas particionadas. Esta es la limitación más importante que distingue el particionamiento nativo de la fragmentación distribuida:

- Las tablas de `lab_bdd` tienen FK entre sí (`pedidos → clientes` , `detalle_pedidos → pedidos` y `detalle_pedidos → productos`). Ninguna puede particionarse directamente sin eliminar primero esas FK.
- En la fragmentación distribuida de las Fases 13–15, la integridad referencial entre fragmentos en nodos distintos se gestiona a nivel de aplicación o mediante scripts de validación periódica. El motor Spider no impone FK entre nodos.

Esta fase crea un esquema separado `lab_particiones` con tablas sin FK para no alterar el esquema `lab_bdd` ya bajo replicación.

7. Mantenimiento de particiones.

Una ventaja clave del particionamiento frente a las tablas normales es la posibilidad de operar sobre particiones individuales sin afectar a las demás:

- `ADD PARTITION` : agrega una partición nueva a una tabla LIST (requiere que el dominio del ENUM ya incluya el nuevo valor) o a una tabla RANGE (requiriendo `REORGANIZE` previo si ya existía `MAXVALUE`).

- `DROP PARTITION` : elimina la partición y **todos sus datos** en una sola operación de metadatos, mucho más rápida que un `DELETE` masivo equivalente sobre InnoDB.
- `REORGANIZE PARTITION p1, p2 INTO (...)` : fusiona o divide particiones existentes redistribuyendo sus filas, sin pérdida de datos.
- `TRUNCATE PARTITION` : vacía los datos de una partición sin eliminar su definición. Útil para archivar datos históricos.
- `ANALYZE PARTITION` : actualiza las estadísticas del optimizador para una o más particiones. Necesario tras inserciones masivas porque InnoDB no actualiza `TABLE_ROWS` en `INFORMATION_SCHEMA` de forma inmediata.

8. Subparticionamiento (*composite partitioning*).

MariaDB permite dividir cada partición en subparticiones usando una segunda estrategia: solo son válidas las combinaciones RANGE+HASH, RANGE+KEY, LIST+HASH y LIST+KEY. El resultado es una tabla con `p × s` segmentos físicos de datos (`p` particiones, `s` subparticiones). Esta fase introduce el concepto con un ejemplo demostrativo; en las Fases 13–15 la distribución entre nodos ofrece mayor flexibilidad y es la ruta recomendada para escalar el laboratorio.

C. Procedimiento paso a paso

Paso 1 — Iniciar las VMs y verificar que la replicación física sigue activa.

Confirmar antes de crear cualquier objeto nuevo: los DDL de `lab_particiones` se propagan a `bdd-nodo02` automáticamente.

Paso 2 — Crear el esquema `lab_particiones`.

Esquema de demostración aislado de `lab_bdd`, sin claves foráneas entre sus tablas.

Paso 3 — Crear y poblar `clientes_list4` : LIST COLUMNS con cuatro particiones.

Una partición por valor de `region`. Caso más directo de particionamiento que mapea exactamente a los fragmentos diseñados en la Fase 9.

Paso 4 — Crear y poblar `clientes_list2` y `pedidos_list2` : LIST COLUMNS con dos particiones.

Agrupar dos regiones por partición (`norte+este` / `sur+oeste`), replicando los predicados exactos de `frag_A` y `frag_B` del Documento de Diseño Distribuido. Son el puente conceptual directo con la Fase 13.

Paso 5 — Crear y poblar `pedidos_range` : RANGE por año.

Demuestra particionamiento temporal, patrón habitual en sistemas con datos históricos que deben archivarse o eliminarse periódicamente.

Paso 6 — Crear `accesos_hash` y `log_eventos_key` : HASH y KEY.

Demuestran distribución uniforme sin predicado de negocio explícito, con columna numérica (HASH) y columna de texto (KEY).

Paso 7 — Verificar la poda de particiones con EXPLAIN.

Ejecutar consultas con y sin predicado sobre el atributo de particionamiento; comparar las columnas `partitions` y `rows` en el plan de ejecución para los cuatro tipos.

Paso 8 — Auditar la distribución de filas con INFORMATION_SCHEMA.PARTITIONS.

Consultar las métricas de cada partición: filas estimadas, tamaño de datos e índices.

Paso 9 — Ejecutar operaciones de mantenimiento de particiones.

Practicar `ADD PARTITION`, `REORGANIZE PARTITION`, `TRUNCATE PARTITION`, `ANALYZE PARTITION` y `DROP PARTITION` sobre las tablas de demostración.

Paso 10 — Demostrar subparticionamiento (LIST + HASH).

Crear `clientes_subpart` con cuatro particiones y dos subparticiones cada una.

Paso 11 — Demostrar la restricción con claves foráneas.

Intentar agregar FK a una tabla particionada y particionar una tabla que ya tiene FK, observando y documentando el error en ambos casos.

Paso 12 — Apagar las VMs y tomar el snapshot `fase12-completa`.

D. Comandos completos

Los comandos marcados **(VM)** se ejecutan en una sesión SSH en la VM indicada.

Los marcados **(host)** se ejecutan en PowerShell en Windows.

Los bloques iniciados con `sudo mariadb` se ejecutan dentro del motor MariaDB.

D.1 Iniciar las VMs y verificar la replicación (host + VM)

PowerShell

```
# Iniciar los tres nodos activos
VBoxManage startvm "bdd-nodo01" --type headless
VBoxManage startvm "bdd-nodo02" --type headless
VBoxManage startvm "bdd-nodo03" --type headless
```

Esperar 25–30 segundos y conectarse a `bdd-nodo01`:

PowerShell

```
ssh bddadmin@192.168.56.101
```

Verificar que la replicación física sigue activa (**VM — bdd-nodo01**):

Bash

```
sudo mariadb -e "SHOW MASTER STATUS\G"
```

La salida debe mostrar un `File` de binlog activo y una `Position` mayor a cero.

Verificar en `bdd-nodo02` que el esclavo sigue funcionando (**nueva terminal PowerShell**):

PowerShell

```
ssh bddadmin@192.168.56.102
```

Bash

```
# En bdd-nodo02
sudo mariadb -e "SHOW REPLICATION STATUS\G" 2>/dev/null || \
sudo mariadb -e "SHOW SLAVE STATUS\G"
```

Buscar `Replica_IO_Running: Yes` y `Replica_SQL_Running: Yes`. Si alguna muestra `No`, resolver el problema de replicación antes de continuar con esta fase.

D.2 Crear el esquema `lab_particiones` (VM — bdd-nodo01)

 Bash

```
sudo mariadb << 'EOF'
-- Esquema dedicado a la demostración de particionamiento nativo.
-- Aislado de lab_bdd para no tocar objetos ya replicados con FK.
CREATE DATABASE IF NOT EXISTS lab_particiones
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci
    COMMENT 'Demostración de particionamiento nativo MariaDB - Fase 12';
-- Confirmar creación
SELECT SCHEMA_NAME,
       DEFAULT_CHARACTER_SET_NAME AS charset,
       DEFAULT_COLLATION_NAME     AS collation
FROM information_schema.SCHEMATA
WHERE SCHEMA_NAME = 'lab_particiones';
EOF
```

Confirmar que el nuevo esquema ya llegó a `bdd-nodo02` (la replicación lo propagó):

 Bash

```
# En bdd-nodo02 - esperar ~5 segundos tras crear el esquema en nodo01
sudo mariadb -e "SHOW DATABASES LIKE 'lab_particiones';"
```

Si aparece `lab_particiones`, la replicación funciona correctamente y se puede continuar.

D.3 LIST COLUMNS — cuatro particiones (una por región) (VM — bdd-nodo01)

 Bash

```
sudo mariadb lab_particiones << 'EOF'
-- =====
-- TIPO: LIST COLUMNS — 4 particiones, una por valor de ENUM
-- Mapeo con Fase 9: cada partición corresponde exactamente
-- a un valor del dominio del atributo de fragmentación.
-- =====
CREATE TABLE IF NOT EXISTS clientes_list4 (
    id            INT            NOT NULL AUTO_INCREMENT,
    nombre        VARCHAR(100) NOT NULL,
    apellido      VARCHAR(100) NOT NULL,
    email         VARCHAR(150),
    telefono      VARCHAR(20),
    region        VARCHAR(10) NOT NULL,
    ciudad        VARCHAR(100),
    fecha_alta    DATETIME DEFAULT CURRENT_TIMESTAMP,
    -- La PK DEBE incluir la columna de particionamiento.
    -- Con PRIMARY KEY(id) solo, MariaDB devuelve ERROR 1503.
    PRIMARY KEY (id, region)
) ENGINE=InnoDB
COMMENT='Demo LIST COLUMNS — 4 particiones, una por región'
PARTITION BY LIST COLUMNS (region) (
    PARTITION p_norte  VALUES IN ('norte'),
    PARTITION p_sur    VALUES IN ('sur'),
    PARTITION p_este   VALUES IN ('este'),
    PARTITION p_oeste  VALUES IN ('oeste')
);
-- Poblar desde lab_bdd (cross-database SELECT, requiere conexión como root)
INSERT INTO clientes_list4
    (id, nombre, apellido, email, telefono, region, ciudad, fecha_alta)
SELECT id, nombre, apellido, email, telefono, region, ciudad, fecha_alta
FROM lab_bdd.clientes;
-- Las estadísticas de InnoDB no se actualizan al instante:
-- ANALYZE TABLE fuerza el recálculo de TABLE_ROWS en INFORMATION_SCHEMA
ANALYZE TABLE clientes_list4;
-- Verificar la distribución de filas por partición
SELECT PARTITION_NAME            AS particion,
       PARTITION_DESCRIPTION     AS region_cubierta,
       TABLE_ROWS               AS filas
FROM information_schema.PARTITIONS
```

```
WHERE TABLE_SCHEMA = 'lab_particiones'
      AND TABLE_NAME = 'clientes_list4'
ORDER BY PARTITION_ORDINAL_POSITION;
-- Resultado esperado: 4 filas, 5 clientes en cada partición
EOF
```

D.4 LIST COLUMNS — dos particiones (predicados del DDD, Fase 9) (VM — bdd-nodo01)

 Bash

```
sudo mariadb lab_particiones << 'EOF'
-- =====
-- TIPO: LIST COLUMNS — 2 particiones
-- Predicados: IDÉNTICOS a frag_A y frag_B del DDD (Fase 9).
-- Esta tabla es el puente conceptual directo con la Fase 13:
-- lo que aquí son particiones locales, allí serán nodos físicos.
-- =====
CREATE TABLE IF NOT EXISTS clientes_list2 (
    id            INT            NOT NULL AUTO_INCREMENT,
    nombre        VARCHAR(100) NOT NULL,
    apellido      VARCHAR(100) NOT NULL,
    email         VARCHAR(150),
    telefono      VARCHAR(20),
    region        VARCHAR(10) NOT NULL,
    ciudad        VARCHAR(100),
    fecha_alta    DATETIME DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id, region)
) ENGINE=InnoDB
COMMENT='Demo LIST COLUMNS — 2 particiones (espeja frag_A y frag_B del DDD
Fase 9) '
PARTITION BY LIST COLUMNS (region) (
    PARTITION frag_A VALUES IN ('norte', 'este'),
    PARTITION frag_B VALUES IN ('sur', 'oeste')
);
INSERT INTO clientes_list2
    (id, nombre, apellido, email, telefono, region, ciudad, fecha_alta)
SELECT id, nombre, apellido, email, telefono, region, ciudad, fecha_alta
FROM lab_bdd.clientes;
-- -----
-- Tabla de pedidos con el mismo esquema de 2 particiones
-- -----
-- IMPORTANTE: NO se define FOREIGN KEY (cliente_id → clientes_list2.id)
-- porque el motor rechaza FK en tablas particionadas. Ver sección D.11.
CREATE TABLE IF NOT EXISTS pedidos_list2 (
    id            INT            NOT NULL AUTO_INCREMENT,
    cliente_id    INT            NOT NULL,
    region        VARCHAR(10) NOT NULL,
    fecha_pedido  DATETIME       DEFAULT CURRENT_TIMESTAMP,
    estado        ENUM('pendiente', 'procesado', 'enviado',
```

```

        'entregado','cancelado') DEFAULT 'pendiente',
    total          DECIMAL(10,2),
    PRIMARY KEY    (id, region)
) ENGINE=InnoDB
COMMENT='Demo pedidos LIST COLUMNS - 2 particiones (espeja frag_A y frag_B
del DDD)'
PARTITION BY LIST COLUMNS (region) (
    PARTITION frag_A VALUES IN ('norte', 'este'),
    PARTITION frag_B VALUES IN ('sur', 'oeste')
);
INSERT INTO pedidos_list2
    (id, cliente_id, region, fecha_pedido, estado, total)
SELECT id, cliente_id, region, fecha_pedido, estado, total
FROM lab_bdd.pedidos;
ANALYZE TABLE clientes_list2;
ANALYZE TABLE pedidos_list2;
-- Verificar distribución en ambas tablas
SELECT 'clientes_list2' AS tabla, PARTITION_NAME AS particion,
    PARTITION_DESCRIPTION AS regiones, TABLE_ROWS AS filas
FROM information_schema.PARTITIONS
WHERE TABLE_SCHEMA = 'lab_particiones'
    AND TABLE_NAME   = 'clientes_list2'
UNION ALL
SELECT 'pedidos_list2', PARTITION_NAME,
    PARTITION_DESCRIPTION, TABLE_ROWS
FROM information_schema.PARTITIONS
WHERE TABLE_SCHEMA = 'lab_particiones'
    AND TABLE_NAME   = 'pedidos_list2'
ORDER BY tabla, particion;
-- Confirmar la integridad con conteos exactos
SELECT 'lab_bdd.clientes (referencia)' AS origen, COUNT(*) AS total
FROM lab_bdd.clientes
UNION ALL
SELECT 'clientes_list4', COUNT(*) FROM clientes_list4
UNION ALL
SELECT 'clientes_list2', COUNT(*) FROM clientes_list2
UNION ALL
SELECT 'lab_bdd.pedidos (referencia)', COUNT(*) FROM lab_bdd.pedidos
UNION ALL
SELECT 'pedidos_list2', COUNT(*) FROM pedidos_list2;
EOF

```

D.5 RANGE — particionamiento por año de pedido (VM — bdd-nodo01)

 Bash

```
sudo mariadb lab_particiones << 'EOF'
-- =====
-- TIPO: RANGE — partición por año
-- Expresión: YEAR(fecha_pedido)
-- Uso típico: datos con dimensión temporal clara; permite
-- archivar o eliminar datos históricos por partición completa,
-- sin DELETE masivo ni impacto en particiones activas.
-- =====
CREATE TABLE IF NOT EXISTS pedidos_range (
    id            INT            NOT NULL AUTO_INCREMENT,
    cliente_id    INT            NOT NULL,
    region        ENUM('norte','sur','este','oeste') NOT NULL,
    fecha_pedido  DATETIME       DEFAULT CURRENT_TIMESTAMP,
    estado        ENUM('pendiente','procesado','enviado',
                      'entregado','cancelado') DEFAULT 'pendiente',
    total         DECIMAL(10,2),
    -- La expresión de particionamiento es YEAR(fecha_pedido).
    -- Como YEAR() no puede incluirse literalmente en la PK,
    -- se incluye la columna base fecha_pedido.
    PRIMARY KEY (id, fecha_pedido)
) ENGINE=InnoDB
COMMENT='Demo RANGE — partición por año de pedido'
PARTITION BY RANGE (YEAR(fecha_pedido)) (
    PARTITION p_anterior  VALUES LESS THAN (2024),
    PARTITION p_2024      VALUES LESS THAN (2025),
    PARTITION p_2025      VALUES LESS THAN (2026),
    PARTITION p_2026      VALUES LESS THAN (2027),
    PARTITION p_futuro    VALUES LESS THAN MAXVALUE
);
-- Insertar los pedidos reales de lab_bdd
INSERT INTO pedidos_range
    (id, cliente_id, region, fecha_pedido, estado, total)
SELECT  id, cliente_id, region, fecha_pedido, estado, total
FROM lab_bdd.pedidos;
-- Añadir pedidos simulados de años anteriores para poblar más particiones
INSERT INTO pedidos_range (cliente_id, region, fecha_pedido, estado,
total) VALUES
( 1, 'norte', '2023-03-15 10:00:00', 'entregado', 3500.00),
( 2, 'norte', '2023-06-20 14:30:00', 'entregado', 1200.00),
( 6, 'sur',   '2023-09-10 09:15:00', 'entregado', 4200.00),
(11, 'este',  '2024-01-05 11:00:00', 'entregado',  950.00),
```

```
(16, 'oeste', '2024-07-22 16:45:00', 'entregado', 2800.00);
ANALYZE TABLE pedidos_range;
SELECT PARTITION_NAME          AS particion,
       PARTITION_DESCRIPTION AS limite_superior,
       TABLE_ROWS             AS filas
FROM information_schema.PARTITIONS
WHERE TABLE_SCHEMA = 'lab_particiones'
     AND TABLE_NAME  = 'pedidos_range'
ORDER BY PARTITION_ORDINAL_POSITION;
-- Consulta por un año específico: la poda debería activarse
SELECT COUNT(*) AS pedidos_2025,
       ROUND(SUM(total), 2) AS facturacion_2025
FROM pedidos_range
WHERE YEAR(fecha_pedido) = 2025;
EOF
```

D.6 HASH y KEY — distribución uniforme (VM — bdd-nodo01)

 Bash

```
sudo mariadb lab_particiones << 'EOF'
-- =====
-- TIPO: HASH — distribución por módulo sobre id (numérico)
-- Uso: distribución uniforme cuando no existe predicado
-- de negocio natural para segmentar.
-- =====

CREATE TABLE IF NOT EXISTS accesos_hash (
    id          BIGINT          NOT NULL AUTO_INCREMENT,
    usuario_id  INT             NOT NULL,
    recurso     VARCHAR(200) NOT NULL,
    accion      ENUM('SELECT','INSERT','UPDATE','DELETE') NOT NULL,
    ts          DATETIME DEFAULT CURRENT_TIMESTAMP,
    ip_origen   VARCHAR(45),
    -- Con HASH(id) y PK(id) la restricción se satisface porque
    -- la expresión de particionamiento depende directamente de la PK.
    PRIMARY KEY (id)
) ENGINE=InnoDB
COMMENT='Log de accesos — demostración HASH por id'
PARTITION BY HASH (id) PARTITIONS 4;
-- Insertar 100 registros de prueba (5 accesos por cada cliente ×
20 clientes)
INSERT INTO accesos_hash (usuario_id, recurso, accion, ip_origen)
SELECT
    c.id                                AS usuario_id,
    CONCAT('/api/recurso/', n.n, '/', c.id) AS recurso,
    ELT(((c.id + n.n - 1) % 4) + 1,
        'SELECT','INSERT','UPDATE','DELETE') AS accion,
    CONCAT('192.168.56.', 100 + (c.id % 7)) AS ip_origen
FROM lab_bdd.clientes c
CROSS JOIN (
    SELECT 1 AS n UNION SELECT 2 UNION SELECT 3
    UNION SELECT 4 UNION SELECT 5
) n;
ANALYZE TABLE accesos_hash;
SELECT PARTITION_NAME AS particion,
       TABLE_ROWS     AS filas_estimadas
FROM information_schema.PARTITIONS
WHERE TABLE_SCHEMA = 'lab_particiones'
      AND TABLE_NAME = 'accesos_hash'
ORDER BY PARTITION_NAME;
-- =====
```

```

-- TIPO: KEY — hashing interno de MariaDB sobre columna de texto
-- Uso: distribución uniforme sobre columnas no numéricas.
-- =====
CREATE TABLE IF NOT EXISTS log_eventos_key (
    id            BIGINT            NOT NULL AUTO_INCREMENT,
    nivel         ENUM('DEBUG','INFO','WARN','ERROR','FATAL') NOT NULL,
    componente    VARCHAR(50)      NOT NULL,
    mensaje       TEXT,
    ts            DATETIME DEFAULT CURRENT_TIMESTAMP,
    servidor      VARCHAR(30),
    -- PK compuesta porque 'componente' es el atributo de particionamiento
    PRIMARY KEY (id, componente)
) ENGINE=InnoDB
COMMENT='Log de eventos — demostración KEY sobre columna de texto'
PARTITION BY KEY (componente) PARTITIONS 4;
INSERT INTO log_eventos_key (nivel, componente, mensaje, servidor) VALUES
('INFO',  'replicacion', 'Binlog position actualizada',
'bdd-nodo01'),
('INFO',  'replicacion', 'Replica IO thread activo',
'bdd-nodo02'),
('WARN',  'conexiones',  'Pool de conexiones al 80%',
'bdd-nodo01'),
('ERROR', 'particion',   'No se encontró partición para valor
recibido','bdd-nodo04'),
('INFO',  'spider',      'Consulta ejecutada en 2 nodos remotos',
'bdd-nodo06'),
('DEBUG', 'consultas',   'Plan de ejecución seleccionado',
'bdd-nodo01'),
('INFO',  'respaldos',   'mysqldump completado exitosamente',
'bdd-nodo01'),
('FATAL', 'replicacion', 'Replica SQL thread detenido por error',
'bdd-nodo02'),
('INFO',  'conexiones',  'Nueva conexión desde 192.168.56.107',
'bdd-nodo06'),
('WARN',  'consultas',   'Slow query detectada: 3.2 segundos',
'bdd-nodo01'),
('INFO',  'particion',   'ANALYZE PARTITION completado',
'bdd-nodo04'),
('ERROR', 'spider',      'Timeout en nodo remoto 192.168.56.105',
'bdd-nodo06');
ANALYZE TABLE log_eventos_key;
SELECT PARTITION_NAME AS particion,
        TABLE_ROWS    AS filas_estimadas
FROM information_schema.PARTITIONS
WHERE TABLE_SCHEMA = 'lab_particiones'

```

```
    AND TABLE_NAME    = 'log_eventos_key'  
ORDER BY PARTITION_NAME;  
EOF
```

D.7 Verificación de poda de particiones con EXPLAIN (VM — bdd-nodo01)

 Bash

```
sudo mariadb lab_particiones << 'EOF'

-- =====
-- EXPERIMENTO A: LIST - poda activa con predicado de igualdad
-- =====
-- Con predicado exacto → EXPLAIN debe mostrar solo 'p_norte'
EXPLAIN SELECT id, nombre, ciudad
FROM clientes_list4
WHERE region = 'norte';
-- Sin predicado → EXPLAIN muestra las cuatro particiones
EXPLAIN SELECT id, nombre, ciudad
FROM clientes_list4;
-- Con IN sobre dos valores → solo las particiones de esos valores
EXPLAIN SELECT id, nombre, ciudad
FROM clientes_list4
WHERE region IN ('norte', 'este');
-- =====
-- EXPERIMENTO B: RANGE - poda temporal
-- =====
-- Solo la partición del año filtrado debe aparecer
EXPLAIN SELECT COUNT(*), SUM(total)
FROM pedidos_range
WHERE YEAR(fecha_pedido) = 2025;
-- Rango de dos años → dos particiones
EXPLAIN SELECT COUNT(*), SUM(total)
FROM pedidos_range
WHERE fecha_pedido BETWEEN '2024-01-01' AND '2025-12-31';
-- Sin filtro temporal → todas las particiones, incluida p_futuro
EXPLAIN SELECT COUNT(*), SUM(total)
FROM pedidos_range;
-- =====
-- EXPERIMENTO C: HASH - la poda NO funciona con predicados de negocio
-- El motor no puede deducir el módulo a partir de usuario_id.
-- =====
-- Todas las particiones se consultan aunque filtremos por usuario_id
EXPLAIN SELECT COUNT(*)
FROM accesos_hash
WHERE usuario_id = 1;
-- Solo con el atributo de hash exacto (id) puede podar
EXPLAIN SELECT * FROM accesos_hash WHERE id = 42;
-- =====
-- RESUMEN: conteo exacto por partición para confirmar distribución
```

```
-- =====
SELECT 'p_norte (list4)' AS particion, COUNT(*) AS filas
FROM clientes_list4 WHERE region = 'norte' UNION ALL
SELECT 'p_sur (list4)', COUNT(*) FROM clientes_list4 WHERE region = 'sur'
UNION ALL
SELECT 'p_este (list4)', COUNT(*) FROM clientes_list4 WHERE region = 'este'
UNION ALL
SELECT 'p_oeste (list4)', COUNT(*) FROM clientes_list4 WHERE region =
'oeste' UNION ALL
SELECT 'frag_A (list2)', COUNT(*) FROM clientes_list2 WHERE region IN
('norte','este') UNION ALL
SELECT 'frag_B (list2)', COUNT(*) FROM clientes_list2 WHERE region
IN ('sur','oeste');
EOF
```

Interpretación de EXPLAIN: la columna `partitions` lista exactamente las particiones que el motor leerá. La columna `rows` indica las filas estimadas a examinar. Compara el valor de `rows` entre la consulta con filtro (`= 'norte'`) y sin filtro (tabla completa) para cuantificar el beneficio de la poda.

D.8 Auditoría con INFORMATION_SCHEMA.PARTITIONS (VM — bdd-nodo01)

 Bash

```
sudo mariadb lab_particiones << 'EOF'
-- Vista completa de todas las particiones del esquema
SELECT
    TABLE_NAME                AS tabla,
    PARTITION_NAME              AS particion,
    PARTITION_METHOD            AS tipo,
    PARTITION_EXPRESSION        AS expresion,
    PARTITION_DESCRIPTION       AS definicion,
    TABLE_ROWS                 AS filas_est,
    ROUND(DATA_LENGTH / 1024.0, 2) AS datos_KB,
    ROUND(INDEX_LENGTH / 1024.0, 2) AS indices_KB
FROM information_schema.PARTITIONS
WHERE TABLE_SCHEMA = 'lab_particiones'
ORDER BY TABLE_NAME, PARTITION_ORDINAL_POSITION;
-- Totales por tabla: útil para comparar contra las referencias de lab_bdd
SELECT
    TABLE_NAME                AS tabla,
    COUNT(PARTITION_NAME)      AS num_particiones,
    SUM(TABLE_ROWS)            AS filas_totales_est,
    ROUND(SUM(DATA_LENGTH + INDEX_LENGTH) / 1024.0, 2) AS total_KB
FROM information_schema.PARTITIONS
WHERE TABLE_SCHEMA = 'lab_particiones'
GROUP BY TABLE_NAME
ORDER BY TABLE_NAME;
EOF
```

D.9 Mantenimiento de particiones (VM — bdd-nodo01)

 Bash

```
sudo mariadb lab_particiones << 'EOF'
-- =====
-- OPERACIÓN 1: ADD PARTITION
-- Escenario: la empresa abre operaciones en 'centro'; se agrega
-- una quinta partición a clientes_list4.
-- =====
-- Paso a: ampliar el ENUM para aceptar el nuevo valor
ALTER TABLE clientes_list4
    MODIFY COLUMN region
        ENUM('norte','sur','este','oeste','centro') NOT NULL;
-- Paso b: crear la partición que recibirá ese valor
ALTER TABLE clientes_list4
    ADD PARTITION (PARTITION p_centro VALUES IN ('centro'));
-- Confirmar la nueva estructura
SELECT PARTITION_NAME, PARTITION_DESCRIPTION, TABLE_ROWS
FROM information_schema.PARTITIONS
WHERE TABLE_SCHEMA = 'lab_particiones'
    AND TABLE_NAME   = 'clientes_list4'
ORDER BY PARTITION_ORDINAL_POSITION;
-- Insertar un cliente de prueba en la nueva partición
INSERT INTO clientes_list4 (id, nombre, apellido, email, region, ciudad)
VALUES (21, 'Gabriela', 'Núñez', 'gabriela.nunez@lab.test',
'centro', 'CDMX');
SELECT region, COUNT(*) AS clientes FROM clientes_list4 GROUP BY region ORDER
BY region;
-- =====
-- OPERACIÓN 2: REORGANIZE PARTITION
-- Escenario: dividir la partición frag_A de clientes_list2
-- en dos particiones individuales (norte y este por separado).
-- =====
ALTER TABLE clientes_list2
    REORGANIZE PARTITION frag_A INTO (
        PARTITION p_norte VALUES IN ('norte'),
        PARTITION p_este  VALUES IN ('este')
    );
ANALYZE TABLE clientes_list2;
SELECT PARTITION_NAME, PARTITION_DESCRIPTION, TABLE_ROWS
FROM information_schema.PARTITIONS
WHERE TABLE_SCHEMA = 'lab_particiones'
    AND TABLE_NAME   = 'clientes_list2'
ORDER BY PARTITION_ORDINAL_POSITION;
```

```

-- El total de clientes_list2 debe seguir siendo 20
SELECT COUNT(*) AS total_clientes_list2 FROM clientes_list2;
-- =====
-- OPERACIÓN 3: TRUNCATE PARTITION
-- Escenario: los pedidos de 2023 ya están en almacenamiento
-- histórico externo; se eliminan del sistema activo.
-- TRUNCATE PARTITION es DDL y es MUCHO más rápido que
-- DELETE FROM ... WHERE YEAR(fecha) < 2024 en tablas grandes.
-- =====
-- Antes: distribución por año
SELECT YEAR(fecha_pedido) AS anyo, COUNT(*) AS pedidos
FROM pedidos_range
GROUP BY anyo ORDER BY anyo;
-- Vaciar solo la partición p_anterior (año < 2024)
ALTER TABLE pedidos_range TRUNCATE PARTITION p_anterior;
ANALYZE TABLE pedidos_range;
SELECT PARTITION_NAME, PARTITION_DESCRIPTION, TABLE_ROWS
FROM information_schema.PARTITIONS
WHERE TABLE_SCHEMA = 'lab_particiones'
  AND TABLE_NAME   = 'pedidos_range'
ORDER BY PARTITION_ORDINAL_POSITION;
-- =====
-- OPERACIÓN 4: REORGANIZE PARTITION de MAXVALUE
-- Escenario: a comienzos de 2027 se necesita una partición
-- explícita para ese año. No se puede ADD directamente porque
-- p_futuro ya cubre MAXVALUE; se reorganiza para insertar p_2027.
-- =====
ALTER TABLE pedidos_range
  REORGANIZE PARTITION p_futuro INTO (
    PARTITION p_2027  VALUES LESS THAN (2028),
    PARTITION p_futuro VALUES LESS THAN MAXVALUE
  );
SELECT PARTITION_NAME, PARTITION_DESCRIPTION
FROM information_schema.PARTITIONS
WHERE TABLE_SCHEMA = 'lab_particiones'
  AND TABLE_NAME   = 'pedidos_range'
ORDER BY PARTITION_ORDINAL_POSITION;
-- =====
-- OPERACIÓN 5: ANALYZE PARTITION – actualizar estadísticas
-- =====
ALTER TABLE clientes_list4 ANALYZE PARTITION ALL;
ALTER TABLE pedidos_range ANALYZE PARTITION p_2024, p_2025, p_2026;
-- =====
-- OPERACIÓN 6: DROP PARTITION
-- Escenario: la expansión al 'centro' fue cancelada.

```

```
-- DROP PARTITION elimina la partición Y TODOS SUS DATOS.
-- =====
-- Cuántos registros perderemos
SELECT COUNT(*) AS filas_a_eliminar FROM clientes_list4 WHERE region
= 'centro';
-- Eliminar la partición junto con sus datos
ALTER TABLE clientes_list4 DROP PARTITION p_centro;
-- El cliente 21 (CDMX) ya no existe
SELECT COUNT(*) AS total_list4 FROM clientes_list4;    -- debe ser 20
EOF
```

D.10 Subparticionamiento — LIST + HASH (VM — bdd-nodo01)

 Bash

```
sudo mariadb lab_particiones << 'EOF'
-- =====
-- SUBPARTICIONAMIENTO: LIST COLUMNS (región) + HASH (id)
-- Combina la granularidad de negocio (región) con distribución
-- interna uniforme (hash). Resultado: 4 × 2 = 8 segmentos.
-- Combinaciones válidas en MariaDB:
--   RANGE+HASH | RANGE+KEY | LIST+HASH | LIST+KEY
-- =====
CREATE TABLE IF NOT EXISTS clientes_subpart (
    id            INT            NOT NULL AUTO_INCREMENT,
    nombre        VARCHAR(100) NOT NULL,
    apellido      VARCHAR(100) NOT NULL,
    region        ENUM('norte', 'sur', 'este', 'oeste') NOT NULL,
    ciudad        VARCHAR(100),
    PRIMARY KEY (id, region)
) ENGINE=InnoDB
COMMENT='Demo subparticionamiento LIST COLUMNS + HASH - 4x2 segmentos'
PARTITION BY LIST COLUMNS (region)
SUBPARTITION BY HASH (id) SUBPARTITIONS 2
(
    PARTITION p_norte  VALUES IN ('norte'),
    PARTITION p_sur    VALUES IN ('sur'),
    PARTITION p_este   VALUES IN ('este'),
    PARTITION p_oeste  VALUES IN ('oeste')
);
INSERT INTO clientes_subpart (id, nombre, apellido, region, ciudad)
SELECT id, nombre, apellido, region, ciudad
FROM lab_bdd.clientes;
ANALYZE TABLE clientes_subpart;
-- Verificar la estructura completa: 8 segmentos de datos
SELECT PARTITION_NAME,
       SUBPARTITION_NAME,
       PARTITION_DESCRIPTION AS region,
       TABLE_ROWS           AS filas_est
FROM information_schema.PARTITIONS
WHERE TABLE_SCHEMA = 'lab_particiones'
      AND TABLE_NAME = 'clientes_subpart'
ORDER BY PARTITION_ORDINAL_POSITION, SUBPARTITION_ORDINAL_POSITION;
EOF
```

Nota pedagógica: el subparticionamiento añade complejidad de administración sin ventajas significativas para los volúmenes de este laboratorio. Se introduce como concepto reconocible en documentación y entornos de producción con tablas de miles de millones de filas. En las Fases 13–15 la distribución entre nodos ofrece mayor flexibilidad para los objetivos del laboratorio.

D.11 Demostración de la incompatibilidad con claves foráneas (VM — bdd-nodo01)

Los dos comandos `ALTER TABLE` de este bloque **fallarán intencionalmente**. Los errores son el resultado esperado y forman parte del aprendizaje. Si la sesión se interrumpe tras el primer error, reconectar con `sudo mariadb` y ejecutar el segundo bloque de forma independiente.

 Bash

```
sudo mariadb << 'EOF'

-- =====
-- CASO 1: Intentar agregar FK a una tabla ya particionada
-- Resultado esperado: ERROR 1506 o similar
-- =====
ALTER TABLE lab_particiones.pedidos_list2
  ADD CONSTRAINT fk_pedidos_list2_cliente
  FOREIGN KEY (cliente_id)
  REFERENCES lab_particiones.clientes_list2(id);
-- =====
-- CASO 2: Intentar particionar una tabla que ya tiene FK
-- lab_bdd.pedidos tiene FK hacia lab_bdd.clientes
-- Resultado esperado: ERROR — la tabla no debe modificarse
-- =====
ALTER TABLE lab_bdd.pedidos
  PARTITION BY LIST COLUMNS (region) (
    PARTITION frag_A VALUES IN ('norte', 'este'),
    PARTITION frag_B VALUES IN ('sur', 'oeste')
  );
EOF
```

Verificar que `lab_bdd.pedidos` sigue sin particionar (el segundo ALTER debe haber fallado sin modificar la tabla):

 Bash

```
sudo mariadb -e "SHOW CREATE TABLE lab_bdd.pedidos\G" | grep -i partition
```

Si no aparece ninguna línea con `PARTITION`, la tabla quedó intacta, que es el resultado correcto.

 Bash

```
# Confirmar también el conteo de filas de lab_bdd.pedidos (debe seguir siendo 20)
sudo mariadb -e "SELECT COUNT(*) AS filas FROM lab_bdd.pedidos;"
```

D.12 Apagar las VMs y tomar los snapshots `fase12-completa` (host)

Desde cada sesión SSH activa, apagar ordenadamente:

 Bash

```
# En bdd-nodo01
sudo poweroff
```

 Bash

```
# En bdd-nodo02
sudo poweroff
```

 Bash

```
# En bdd-nodo03
sudo poweroff
```

Confirmar desde el host que las tres VMs se detuvieron:

 PowerShell

```
VBoxManage list runningvms
```

La salida debe estar vacía. Tomar los snapshots:

 PowerShell

```
VBoxManage snapshot "bdd-nodo01" take "fase12-completa" `
  --description "lab_particiones creado: clientes_list4, clientes_list2,
pedidos_list2, pedidos_range, accesos_hash, log_eventos_key, clientes_subpart.
Poda verificada. Mantenimiento de particiones demostrado. Restriccion
FK demostrada."
VBoxManage snapshot "bdd-nodo02" take "fase12-completa" `
  --description "lab_particiones replicado desde bdd-nodo01 via replicacion
fisica. Contenido identico. Snapshot de hito de fase."
VBoxManage snapshot "bdd-nodo03" take "fase12-completa" `
  --description "lab_particiones replicado desde bdd-nodo01 via replicacion
logica. Snapshot de hito de fase."
```

Confirmar que los snapshots existen:

 PowerShell

```
VBoxManage snapshot "bdd-nodo01" list
VBoxManage snapshot "bdd-nodo02" list
VBoxManage snapshot "bdd-nodo03" list
```

`bdd-nodo01` y `bdd-nodo02` deben mostrar `fase05-completa` hasta `fase12-completa`.
`bdd-nodo03` debe mostrar `fase11-completa` y `fase12-completa` (se creó en la Fase 11).

E. Verificación de funcionamiento

Esta fase se considera completa cuando se cumplen todos los puntos siguientes en `bdd-nodo01`:

1. `SHOW DATABASES;` incluye `lab_particiones` en la lista.
 2. `SHOW TABLES IN lab_particiones;` devuelve exactamente siete tablas: `accesos_hash`, `clientes_list2`, `clientes_list4`, `clientes_subpart`, `log_eventos_key`, `pedidos_list2`, `pedidos_range`.
 3. `INFORMATION_SCHEMA.PARTITIONS` para `clientes_list4` muestra cuatro particiones (`p_norte`, `p_sur`, `p_este`, `p_oeste`) con 5 filas cada una (confirmado con `ANALYZE TABLE` previo).
 4. `INFORMATION_SCHEMA.PARTITIONS` para `clientes_list2` muestra tres particiones tras el `REORGANIZE PARTITION`: `p_norte` (5 filas), `p_este` (5 filas) y `frag_B` (10 filas). Total: 20 filas.
 5. `INFORMATION_SCHEMA.PARTITIONS` para `pedidos_range` muestra la partición `p_anterior` con 0 filas tras el `TRUNCATE PARTITION`.
 6. `EXPLAIN SELECT * FROM clientes_list4 WHERE region = 'norte'` muestra `partitions: p_norte` — poda activa, solo una partición leída.
 7. `EXPLAIN SELECT * FROM clientes_list4` (sin filtro) muestra las cuatro particiones — sin poda, comportamiento esperado.
 8. `EXPLAIN SELECT ... FROM pedidos_range WHERE YEAR(fecha_pedido) = 2025` muestra `partitions: p_2025` — poda temporal activa.
 9. `EXPLAIN SELECT ... FROM accesos_hash WHERE usuario_id = 1` muestra las **cuatro** particiones — confirma que HASH no realiza poda con predicados de negocio distintos al atributo de hash.
 10. `clientes_subpart` muestra 8 filas en `INFORMATION_SCHEMA.PARTITIONS` (4 particiones × 2 subparticiones) con `SUBPARTITION_NAME` distinto de NULL.
 11. El intento de agregar FK a `pedidos_list2` produce un error (`ERROR 1506` o similar) y la tabla no es modificada.
 12. El intento de particionar `lab_bdd.pedidos` produce un error y la tabla original permanece intacta con sus 20 filas y sus FK definidas (verificado con `SHOW CREATE TABLE lab_bdd.pedidos`).
 13. `lab_bdd.clientes` y `lab_bdd.pedidos` en `bdd-nodo01` siguen sin particionamiento y con sus datos originales íntegros.
 14. El esquema `lab_particiones` con sus siete tablas existe también en `bdd-nodo02` (propagado vía replicación física).
 15. Los snapshots `fase12-completa` existen en los tres nodos.
-

F. Problemas comunes y soluciones

Problema	Causa probable	Solución
<pre>CREATE TABLE</pre> falla con <pre>ERROR 1503: A PRIMARY KEY must include all columns in the table's partitioning function</pre>	La clave primaria no incluye la columna de particionamiento	Cambiar la PK a compuesta: <pre>PRIMARY KEY (id, region)</pre> . Con <pre>PARTITION BY HASH(id)</pre> la PK puede ser solo <code>id</code> porque la expresión depende directamente de ella
<pre>ERROR 1526:</pre> Table has no partition for value from column_list al insertar	La tabla destino no tiene una partición que cubra ese valor del dominio	Verificar con <pre>SELECT DISTINCT region FROM lab_bdd.clientes</pre> que todos los valores del origen tienen partición correspondiente en el destino; usar <pre>ADD PARTITION</pre> si falta alguno
<pre>EXPLAIN</pre> no muestra la columna <pre>partitions</pre>	La consulta no apunta a una tabla particionada, o se está usando un cliente que no muestra todas las columnas	En MariaDB 10.x <pre>EXPLAIN</pre> incluye automáticamente e <pre>partitions</pre> ; usar <pre>EXPLAIN PARTITIONS SELECT ...</pre> como sintaxis alternativa compatible
<pre>TABLE_ROWS</pre> en <pre>INFORMATION_SCHEMA.PARTITION</pre>	InnoDB no actualiza las estadísticas de	Ejecutar <pre>ANALYZE TABLE nombre_tabla;</pre>

<code>S</code> muestra 0 o valores incorrectos tras insertar datos	forma inmediata	y volver a consultar <pre> INFORMATION_SCHEMA HEMA.PARTITION S </pre>
El intento de agregar FK a una tabla particionada devuelve <code>ERROR 1217</code> en lugar de <code>1506</code>	Las versiones de MariaDB difieren en el código exacto de error para la combinación FK + particionamiento	Ambos códigos son válidos; lo importante es que el <code>ALTER TABLE</code> falla y la tabla no se modifica
<pre> ALTER TABLE ... REORGANIZE PARTITION </pre> tarda más de lo esperado	MariaDB reconstruye físicamente las particiones afectadas (lectura + escritura completa + reindexado)	Es comportamiento correcto; para el laboratorio con 20 filas no tarda más de 1–2 segundos. En producción con millones de filas se haría en una ventana de mantenimiento
<pre> ADD PARTITION p_centro falla con Partition column values of incorrect type </pre>	El ENUM de la tabla aún no incluye el valor <code>'centro'</code>	Ejecutar primero <pre> ALTER TABLE clientes_list4 MODIFY COLUMN region ENUM('norte',' sur','este','o este','centro') NOT NULL; y luego el ADD PARTITION </pre>
<pre> REORGANIZE PARTITION p_futuro INTO (...) falla con </pre>	Ya existe una partición con el nombre que se quiere crear en el INTO	Elegir un nombre distinto en la cláusula <code>INTO</code>; el nombre de la partición destino no

<code>Duplicate partition name</code>		puede coincidir con ninguna partición existente
El esquema <code>lab_particiones</code> no aparece en <code>bdd-nodo02</code>	La replicación tiene un error pendiente que detiene al SQL thread	En <code>bdd-nodo02</code> : <code>SHOW REPLICAS</code> — buscar <code>Last_SQL_Error</code> ; resolver el error específico antes de continuar
<code>TRUNCATE PARTITION</code> falla con <code>Access denied</code>	El usuario conectado no tiene el privilegio <code>DROP</code> (necesario para <code>TRUNCATE PARTITION</code>)	Conectar con <code>sudo mariadb</code> (root) o conceder: <code>GRANT DROP ON lab_particiones.* TO 'lab_admin'@'localhost';</code> <code>FLUSH PRIVILEGES;</code>
El segundo <code>ALTER TABLE lab_bdd.pedido ... PARTITION BY ...</code> modifica la tabla en lugar de fallar	La versión de MariaDB instalada tiene un comportamiento diferente con esta combinación	Verificar inmediatamente con <code>SHOW CREATE TABLE lab_bdd.pedidos\G</code> ; si la tabla fue particionada, restaurar desde el snapshot <code>fase11-completa</code> de <code>bdd-nodo01</code>

G. Checklist de validación

- ☐ `SHOW DATABASES;` incluye `lab_particiones` en `bdd-nodo01`.

- ☐ `SHOW TABLES IN lab_particiones;` devuelve las siete tablas de la fase.
- ☐ `clientes_list4` tiene cuatro particiones con 5 filas cada una.
- ☐ `clientes_list2` tiene tres particiones tras el `REORGANIZE ( p_norte , p_este , frag_B )` con 20 filas en total.
- ☐ `pedidos_list2` tiene dos particiones (`frag_A` y `frag_B`) con 10 pedidos cada una.
- ☐ `pedidos_range` tiene cinco particiones; `p_anterior` está vacía tras `TRUNCATE PARTITION .`
- ☐ `accesos_hash` tiene cuatro particiones con distribución aproximadamente uniforme entre ~25 filas cada una.
- ☐ `clientes_subpart` muestra 8 segmentos en `INFORMATION_SCHEMA.PARTITIONS` (4 particiones × 2 subparticiones) con `SUBPARTITION_NAME` no nulo.
- ☐ `EXPLAIN SELECT * FROM clientes_list4 WHERE region = 'norte'` muestra `partitions: p_norte` únicamente — poda activa.
- ☐ `EXPLAIN SELECT * FROM clientes_list4` sin filtro muestra las cuatro particiones — sin poda.
- ☐ `EXPLAIN SELECT ... FROM pedidos_range WHERE YEAR(fecha_pedido) = 2025` muestra `partitions: p_2025` únicamente.
- ☐ `EXPLAIN SELECT ... FROM accesos_hash WHERE usuario_id = 1` muestra las cuatro particiones — confirma ausencia de poda con HASH.
- ☐ El intento de agregar FK a `pedidos_list2` produjo un error; la tabla no fue modificada.
- ☐ El intento de particionar `lab_bdd.pedidos` produjo un error; la tabla original sigue intacta con 20 filas y sus FK definidas.
- ☐ `lab_bdd.clientes` y `lab_bdd.pedidos` no tienen particionamiento y conservan sus datos originales.
- ☐ La replicación en `bdd-nodo02` sigue activa (`Replica_IO_Running: Yes`).
- ☐ `lab_particiones` con sus siete tablas existe también en `bdd-nodo02` .
- ☐ Los snapshots `fase12-completa` existen en los tres nodos.
- ☐ Puedo explicar la diferencia técnica entre `RANGE COLUMNS` y `RANGE` estándar (tipos de datos soportados).
- ☐ Puedo explicar por qué `HASH` no realiza poda con predicados de negocio sobre columnas distintas al atributo de particionamiento.
- ☐ Puedo explicar por qué las FK son incompatibles con el particionamiento nativo y cómo las Fases 13–15 abordan esa limitación.

- ☐ Puedo relacionar los predicados de `clientes_list2` / `pedidos_list2` con los fragmentos `frag_A` y `frag_B` del Documento de Diseño Distribuido de la Fase 9.
-

Preguntas teóricas para estudiantes

1. En el experimento de poda de particiones (sección D.7), la consulta sobre `accesos_hash` con `WHERE usuario_id = 1` lee las cuatro particiones a pesar del filtro. Sin embargo, `WHERE id = 42` sobre la misma tabla lee solo una partición. Explica técnicamente por qué el predicado sobre `id` habilita la poda mientras que el predicado sobre `usuario_id` no, aunque ambos son condiciones de igualdad sobre columnas de la misma tabla.
2. La restricción de que el atributo de particionamiento debe estar en la clave primaria tiene una consecuencia sobre el comportamiento del `AUTO_INCREMENT`: en `clientes_list4` con `PRIMARY KEY (id, region)`, dos filas en particiones distintas podrían tener el mismo valor de `id`. Explica cuándo ocurre esto, en qué circunstancias puede ser un problema para la aplicación que consume los datos y qué estrategias se usan en producción para evitar colisiones de identificadores al distribuir datos entre nodos.
3. Compara `ALTER TABLE pedidos_range TRUNCATE PARTITION p_anterior` con un `DELETE FROM pedidos_range WHERE YEAR(fecha_pedido) < 2024`. Ambos producen el mismo resultado en el contenido de la tabla, pero su impacto en el motor es radicalmente distinto. Explica las diferencias en términos de: (a) tipo de operación (DDL vs. DML), (b) generación de binlog y propagación por replicación, © impacto en el tablespace de InnoDB, y (d) tiempo de ejecución esperado para una tabla con 500 millones de filas históricas.
4. El diseño de `clientes_list4` (cuatro particiones, una por región) y el de `clientes_list2` (dos particiones, dos regiones por partición) corresponden a dos estrategias distintas de fragmentación que se verán también en la Fase 13. Discute las ventajas e inconvenientes de cada estrategia en términos de: (a) granularidad de la poda al filtrar por una sola región, (b) balanceo de carga entre particiones o nodos, y © coste de añadir un nuevo valor al dominio del atributo de particionamiento (por ejemplo, una quinta región).
5. La incompatibilidad entre claves foráneas y particionamiento nativo refleja una tensión fundamental entre integridad referencial (garantizada por el motor) e integridad de distribución (garantizada por el esquema de particionamiento). Explica cómo la arquitectura del laboratorio transita de un modelo con FK nativas

(`lab_bdd` , donde la integridad la garantiza InnoDB) a un modelo sin FK (`lab_particiones` y, más adelante, los fragmentos de las Fases 13–15, donde la integridad se gestiona a nivel de aplicación). ¿Cuáles son los riesgos concretos de este cambio de responsabilidad?

Ejercicios prácticos

1. Análisis de selectividad de predicados.

Para la tabla `clientes_list4` , escribir y ejecutar cuatro consultas `EXPLAIN` :

(a) filtro por un solo valor de región (`= 'norte'`), (b) filtro por dos valores (`IN ('norte', 'este')`), © negación (`NOT IN ('sur')`), y (d) sin filtro.

Para cada una, registrar el valor de la columna `partitions` y el valor estimado de `rows` . Elaborar una tabla comparativa y concluir: ¿la reducción de particiones leídas es proporcional a la reducción en el valor de `rows` ?

2. Particionamiento RANGE por precio de producto.

Crear en `lab_particiones` una tabla `productos_range` que copie la estructura de `lab_bdd.productos` (sin FK) y la particione por `RANGE` usando el campo `precio` con tres rangos:

- `p_economico` : precio < 500
- `p_medio` : precio entre 500 y 2 000
- `p_premium` : precio ≥ 2 000

Poblar desde `lab_bdd.productos` , ejecutar `ANALYZE TABLE` , y verificar cuántos productos caen en cada rango con conteos exactos. Ejecutar `EXPLAIN` para confirmar la poda con `WHERE precio < 500` y con `WHERE precio BETWEEN 500 AND 2000` .

Discutir si este particionamiento produciría un buen balanceo en un catálogo de e-commerce real con miles de productos.

3. Verificación de integridad entre tablas particionadas relacionadas.

Las tablas `clientes_list2` y `pedidos_list2` no tienen FK entre sí, pero comparten `cliente_id` . Escribir una consulta SQL que detecte registros huérfanos en `pedidos_list2` : pedidos cuyo `cliente_id` no existe en `clientes_list2` .

Luego insertar deliberadamente un pedido con `cliente_id = 999` (que no existe en clientes) para crear un huérfano, verificar que la consulta de detección lo encuentra, y finalmente eliminarlo. Discutir: ¿qué mecanismo sustituye a las FK en un sistema distribuido real para mantener este tipo de integridad referencial?

Reto adicional para alumnos avanzados

Diseñar e implementar en `lab_particiones` una tabla `pedidos_hibrida` que combine subparticionamiento con **RANGE por semestre** en el nivel superior y **KEY por** `cliente_id` en el nivel inferior:

- Nivel RANGE: dos particiones semestrales para 2025 (`p_sem1_2025` para enero-junio, `p_sem2_2025` para julio-diciembre) más una partición `p_futuro` con `MAXVALUE` .
- Nivel KEY: 3 subparticiones dentro de cada partición semestral (por `cliente_id`).

El resultado debe ser una tabla con $3 \times 3 = 9$ segmentos de datos totales. Poblirla con los 20 pedidos de `lab_bdd.pedidos` más al menos 30 registros adicionales con fechas distribuidas entre los dos semestres. Verificar con `INFORMATION_SCHEMA.PARTITIONS` que los 9 segmentos existen y que los totales de filas son coherentes. Ejecutar `EXPLAIN` para una consulta que filtre por semestre (por ejemplo, `WHERE fecha_pedido BETWEEN '2025-01-01' AND '2025-06-30'`) y confirmar que la poda actúa al nivel del semestre (RANGE) aunque no al nivel de subpartición (KEY). Redactar un párrafo justificando en qué escenario real valdría la pena esta complejidad de administración frente a una solución de dos nodos físicamente separados como se implementará en la Fase 13.

Criterios de evaluación para el profesor

Criterio	Peso	Indicador de logro
Creación de tablas con los cuatro tipos de particionamiento	25%	Las siete tablas existen con las definiciones correctas; el estudiante puede justificar qué tipo de particionamiento se eligió para cada caso de uso y por qué
Verificación de poda de particiones	20%	Los <code>EXPLAIN</code> muestran los valores esperados en la columna <code>partitions</code> para cada tipo; el estudiante interpreta la columna <code>rows</code> y puede explicar cuantitativamente el beneficio de la poda
Operaciones de mantenimiento	20%	Se ejecutaron correctamente <code>REORGANIZE</code> , <code>TRUNCATE</code> , <code>ADD</code> , <code>ANALYZE</code> y <code>DROP PARTITION</code> ; el estudiante puede describir el impacto de cada operación en los datos y en la replicación

Comprensión de la restricción FK	15%	El estudiante puede demostrar el error con evidencia del intento fallido, explicar la causa técnica y articular cómo las Fases 13–15 abordan esa limitación a nivel de aplicación
Conexión con el DDD (Fase 9)	10%	El estudiante relaciona explícitamente los predicados de <code>clientes_list</code> 2 / <code>pedidos_list</code> con los fragmentos <code>frag_A</code> / <code>frag_B</code> del Documento de Diseño Distribuido, y articula la diferencia entre una partición local (Fase 12) y un fragmento en nodo separado (Fase 13)
Comprensión conceptual (preguntas teóricas)	10%	Las respuestas usan vocabulario técnico correcto (poda, DDL, DML, consistencia referencial, AUTO_INCREMENT por partición) y

	hacen referencia explícita al esquema del laboratorio
--	--

Preparación para la siguiente fase

La **Fase 13: Fragmentación Horizontal** requerirá:

- El Documento de Diseño Distribuido (`C:\LabBDD\Documentacion\fase09-disenyo-distribuido.md`) como referencia técnica de los predicados de fragmentación: los mismos `region IN ('norte','este')` y `region IN ('sur','oeste')` que en esta fase definieron particiones locales pasarán a definir fragmentos en nodos físicos separados.
- Comprensión sólida de `LIST COLUMNS` y de la poda de particiones adquirida en esta fase: en la Fase 13 se verá que el motor Spider del coordinador aplica internamente el mismo concepto de predicado de selección para enrutar cada consulta al shard correcto.
- El esquema `lab_bdd` intacto y replicado entre `bdd-nodo01` y `bdd-nodo02`.
- Snapshot `fase12-completa` tomado en los tres nodos.
- **Dos nuevas máquinas virtuales** que deben crearse al inicio de la Fase 13: `bdd-nodo04` (192.168.56.104, fragmento A) y `bdd-nodo05` (192.168.56.105, fragmento B). Se crearán siguiendo el mismo procedimiento de instalación de las Fases 3–7 pero de forma compacta, al ser la tercera y cuarta VM del laboratorio.

En la Fase 13 las tablas `clientes`, `pedidos` y `detalle_pedidos` de `lab_bdd` se cargarán en `bdd-nodo04` y `bdd-nodo05` con los datos ya segmentados según los predicados del DDD. El coordinador (`bdd-nodo06`) no se creará hasta la Fase 14; en la Fase 13 se accederá directamente a cada shard para verificar los datos y las consultas de reconstrucción (`UNION ALL`) se ejecutarán manualmente desde `bdd-nodo01`.
